Kerasnip: A Bridge Between Keras and Tidymodels

by David Díaz

Abstract The kerasnip R package bridges the keras3 R package and the tidymodels framework, allowing users to define Keras models as parsnip model specifications. This seamless integration enables the use of Keras models within the tidymodels ecosystem for preprocessing, tuning, and evaluation. kerasnip dynamically generates parsnip specifications from user-defined layer blocks, simplifying the process of building and tuning complex Keras models. This enables the definition of non-linear, functional architectures by providing a clear way to map connections between layers. This article introduces the package's architecture and demonstrates its use with a case study.

1 Introduction

Deep learning (**?**) has reshaped applied machine learning, and Keras [@keras3; **?**] remains one of the most popular APIs for composing modern neural networks. In R, the keras3 package faithfully mirrors the Keras experience, while the tidymodels framework (**?**) has become the de facto standard interface for preprocessing, tuning, and evaluating models. Although parsnip (a tidymodels component) ships with a basic Keras engine, it only covers a handful of predefined multilayer perceptron templates.

The difficulty emerges when practitioners need to move beyond those templates. Keras models are constructed by freely composing layers, whereas parsnip specifications expect a fixed set of hyperparameters that map directly to arguments. This mismatch prevents R users from expressing arbitrary Keras architectures, repeating blocks, or wiring multi-branch graphs inside a tidymodels workflow.

kerasnip addresses this impedance mismatch by generating parsnip specifications on the fly from user-defined Keras blocks. The package surfaces the entire tidymodels toolchain—recipes, tune, workflows, and workflowsets—for any model that can be described with the Keras sequential or functional APIs. Functional pipelines leverage the `inp_spec()` helper to declare connections explicitly, keeping even complex graphs readable. The package name itself blends **keras** and **parsnip**, underscoring this bridging goal.

The remainder of the paper is structured as follows. Section 2 positions kerasnip relative to existing tooling and summarizes its main advantages. Section 3 details the code generation approach that connects Keras blocks with parsnip. Section 4 walks through a quick-start example, and Section 5 presents a full case study on the Ames Housing data. The paper concludes by outlining the current limitations and near-term roadmap.

2 Ecosystem context and advantages

2.1 Related Work

The kerasnip package builds upon a rich ecosystem of packages for deep learning and machine learning in R. This section provides an overview of the most relevant packages and positions kerasnip in relation to them.

reticulate: The reticulate package (**?**) is the foundation for R-Python interoperability. It allows R users to call Python code from R, and it is the package that the keras3 package uses to interface with the Python Keras library. While reticulate is essential for using keras3 in R, it does not provide any direct integration with the tidymodels framework.

keras3: The keras3 package [@keras3] provides a direct R interface to the Keras library, allowing R users to define and train Keras models using a familiar R syntax. While parsnip offers a basic Keras engine, the keras3 package itself does not provide a native mechanism for integrating custom or dynamically defined Keras architectures into tidymodels workflows, which is the specific challenge kerasnip addresses. Keras offers a high-level, user-friendly API, widespread adoption, and multi-backend support (TensorFlow (**?**), JAX (**?**), PyTorch (**?**)), making it a powerful and versatile choice. kerasnip thus serves as a valuable bridge for users preferring the Keras ecosystem.

luz and brulee: The luz (**?**) and brulee packages provide a tidymodels-like interface for the torch deep learning framework. They are excellent tools for R users who want to use torch with tidymodels. kerasnip works by extending the existing parsnip `keras` engine, enabling a similar level of integration

and flexibility for models built with the keras3 **R package** as luz and brulee do for torch models.

While both kerasnip and luz/brulee integrate deep learning frameworks with tidymodels, they cater to different backends and user preferences. luz and brulee offer an R-native experience with torch, potentially providing lower overhead and direct control. In contrast, kerasnip leverages the high-level Keras API, built on optimized backends (TensorFlow, JAX, PyTorch), offering user-friendliness and rapid prototyping for those familiar with Keras. While luz/brulee might offer more granular control, kerasnip prioritizes abstraction and ease of use. Crucially, kerasnip uniquely enables dynamic definition and architectural tuning of Keras models directly within tidymodels, a capability not fully offered by other existing solutions.

A key design divergence in kerasnip compared to luz/brulee is its emphasis on dynamic model generation through user-defined `layer_blocks` and the `inp_spec()` helper. While luz/brulee provide a tidymodels-like interface for predefined torch modules, kerasnip empowers users to construct arbitrary Keras architectures on the fly, directly within the parsnip specification. This architectural flexibility, particularly the ability to tune the *number* of layers or define complex functional connections via `inp_spec()`, is a deliberate choice to leverage Keras's compositional nature and extend tidymodels' tuning capabilities beyond just hyperparameters to the model's very structure. This approach allows for a more comprehensive architectural search within the tidymodels framework, which is a unique offering of kerasnip.

kerasnip extends the parsnip ecosystem not by adding a new engine, but by enabling the creation of dynamic parsnip model specifications that use the existing `keras` engine. This is important to distinguish from parsnip's default Keras integration, such as `parsnip::keras_mlp()`, which offers limited architectural flexibility. While `keras_mlp()` is convenient for predefined MLP architectures, kerasnip's core strength is its ability to dynamically generate specifications from user-defined `layer_blocks`. This allows users to integrate custom and complex model architectures into tidymodels workflows and enables comprehensive architectural search and tuning (e.g., varying the number of layers or defining complex functional connections), making it powerful for optimizing the very structure of a Keras model, not just its hyperparameters.

In summary, while several packages provide access to deep learning libraries from R, kerasnip is unique in its focus on providing a dedicated, flexible, and tidymodels-native solution for keras3 models. It is designed to be a lightweight and intuitive bridge that allows R users to seamlessly integrate the power of keras3 into their tidymodels workflows, particularly benefiting those already invested in the Keras ecosystem or requiring Keras-specific features not available in other frameworks. Its block-based architecture empowers users to define and tune complex deep learning models with minimal boilerplate, making advanced deep learning techniques more accessible within the familiar tidymodels paradigm.

2.2 Key Advantages of kerasnip

kerasnip offers several practitioner-oriented advantages that complement the ecosystem context above:

- **Reduced boilerplate and full workflow parity:** dynamically generated parsnip specifications eliminate bespoke wrappers. Any generated model can immediately plug into recipes, tune, workflows, or workflowsets, so Keras users receive the exact same ergonomics that tidymodels users expect for glmnet, xgboost, or torch engines.
- **Architectural tuning, not just hyperparameters:** arguments of the form `num_{block_name}` control how many times a block is repeated, enabling structural search in addition to parameter search. Figure "Conceptual diagram illustrating num block tuning" visualizes how varying `num_dense` alters the network depth, and the accompanying code sample shows how to expose `num_dense` to `tune()`. Repeated blocks intentionally share the same hyperparameters; to tune each hidden layer independently, users can register separate block functions (e.g., `dense_head`, `dense_tail`) and assign distinct tuning parameters to each.
- **Extensibility to advanced architectures:** because layer blocks can wrap any Keras layer, the current release already supports CNNs, RNNs, Transformers, or residual graphs—users simply describe the building blocks they need. Future work will focus on shipping higher-level templates and validation helpers for these architectures, not on unlocking the underlying capability.
- **Enhanced reproducibility and performance:** encapsulating the architecture definition in R functions makes it straightforward to version-control both data preprocessing and network topology, while the heavy computation is still executed by optimized TensorFlow, JAX, or PyTorch backends.
- **Debugging support:** helpers such as `compile_keras_grid()` summarize the dynamically generated graphs, and every fitted specification can expose the underlying Keras model for inspection

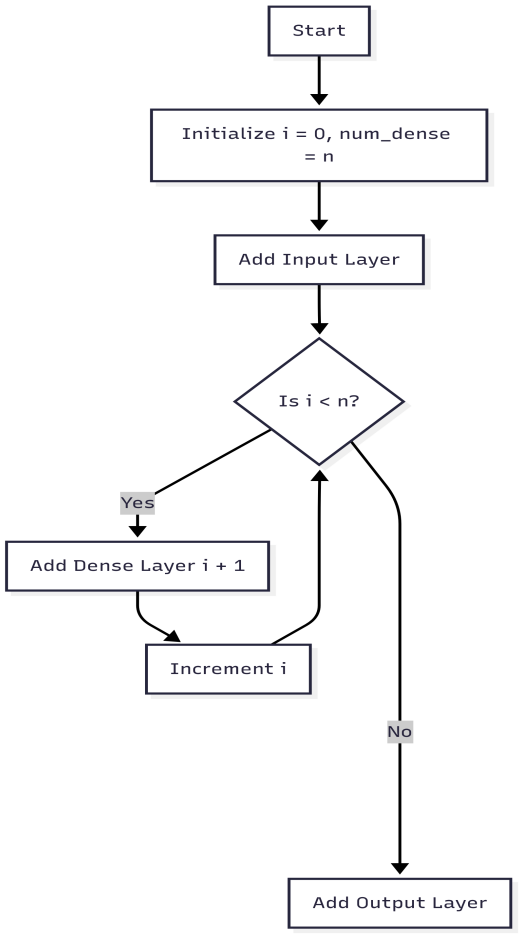

Figure 1: Conceptual diagram illustrating num block tuning

via `summary()` or `plot()`.

```
# Define a tunable model specification
tune_spec <- tunable_mlp(
  num_dense = tune(),
  dense_units = tune(),
  fit_epochs = 10
) |>
  set_engine("keras")

# Define the tuning grid
params <- extract_parameter_set_dials(tune_spec) |>
  update(
    num_dense = dials::num_terms(c(1, 3)),
    dense_units = dials::hidden_units(c(8, 64))
  )
grid <- grid_regular(params, levels = 3)

# Run the tuning
# ...
```

3 Bridging the gap: The kerasnip architecture

The core of **kerasnip** is a code generation engine that bridges the gap between the flexible, layer-based definition of **keras3** models and the structured, argument-based definition of **parsnip** models. It achieves this by dynamically creating a new **parsnip** model specification from a set of user-defined "layer blocks".

When a user calls `create_keras_sequential_spec()` to create a Keras model using the sequential API or `create_keras_functional_spec()` to create a model using the functional API, **kerasnip** inspects the functions provided in the `layer_blocks` argument to identify their formal arguments. It then generates a new function that represents the **parsnip** model specification. This new function has a unified set of arguments that can be used to control the architecture and hyperparameters of the **keras3** model.

To illustrate this, let's consider a simplified example of the code that **kerasnip** generates for a sequential MLP. When you call `create_keras_sequential_spec("my_mlp", ...)` with `input`, `dense`, and `output` layer blocks, **kerasnip** generates a function similar to this:

```
my_mlp <- function(mode = "regression", num_dense = 1, dense_units = 8, ...) {
  args <- list(
    num_dense = num_dense,
    dense_units = dense_units,
    ...
  )
  new_model_spec(
    "my_mlp",
    args = args,
    eng_args = NULL,
    mode = mode,
    engine = "keras"
  )
}
```

This generated function, `my_mlp()`, is a **parsnip** model specification. It has arguments to control the number of dense layers (`num_dense`) and the number of units in each dense layer (`dense_units`). When you call this function, it returns a **parsnip** model specification that can be used in a **tidymodels** workflow.

kerasnip uses a simple naming convention to map the arguments of the generated function to the arguments of the layer blocks. For example, an argument named `dense_units` in the `my_mlp()` function is mapped to the `units` argument of the `dense_block` function. This allows for a flexible and intuitive way to control the architecture of the **keras3** model from the **parsnip** model specification.

For functional API models, **kerasnip** provides the `inp_spec()` helper function to define the connections between layers. You use it to specify which layer outputs (tensors) should be fed into which arguments of the next layer's function. This explicit mapping is crucial for building non-linear, multi-input/multi-output architectures where the flow of data is not a simple sequential chain.

3.1 Using `inp_spec()`

The `inp_spec()` function takes two main arguments:

1. The layer block function to be called (e.g., `concat_block`).
2. A character vector specifying the inputs. The **names** of the vector must match the argument names of the layer block function (e.g., `a` and `b` in `concat_block`), and each value refers to the upstream layer block whose tensor should be supplied to that argument (e.g., `input_a` and `input_b`).

To illustrate, consider a simple functional model with two inputs that are concatenated and then passed to a dense layer:

```
input_a_block <- function(input_shape) {
  layer_input(shape = input_shape, name = "input_a")
}
input_b_block <- function(input_shape) {
  layer_input(shape = input_shape, name = "input_b")
}
concat_block <- function(a, b) {
  layer_concatenate(list(a, b))
}
output_block <- function(tensor) {
  layer_dense(tensor, units = 1)
}
```

```
create_keras_functional_spec(
  "my_functional_model",
  list(
    input_a = input_a_block,
    input_b = input_b_block,
    concatenated = inp_spec(concat_block, c(a = "input_a", b = "input_b")),
    output = inp_spec(output_block, "concatenated")
  )
)
```

In this example:

- `inp_spec(concat_block, c(a = "input_a", b = "input_b"))` specifies that the `concat_block` function should be called.
- The output of the `input_a` block will be passed to the `a` argument of `concat_block`.
- The output of the `input_b` block will be passed to the `b` argument of `concat_block`.
- If the input is a single tensor, you can simply provide the name of the upstream block as a character string, as in `inp_spec(output_block, "concatenated")`.

This explicit way of defining connections allows for the creation of complex, non-linear model architectures.

Clarity on `inp_spec()` for Complex Scenarios

`inp_spec()` is essential for creating complex functional Keras models, enabling intricate data flows like parallel branches or residual connections (e.g., in ResNets). For a residual connection, `inp_spec()` directs tensors from both the main path and a skip connection into a subsequent block, such as one using `layer_add()`. This explicit mapping of outputs to inputs grants fine-grained control, facilitating highly customized and complex network topologies.

Internally, **kerasnip** parses the `inp_spec()` calls to understand the intended data flow and connections between layer blocks. It then leverages `rlang::exec()` and the `!!!` operator for dynamic function execution, enabling the flexible construction of Keras models from user-defined blocks, effectively translating the `inp_spec()` graph into the Keras functional API.

4 A quick start example

To quickly illustrate the core functionality of **kerasnip**, a simple example of a sequential model for classifying the `iris` dataset is presented.

First, the layer blocks for the model are defined:

```
# Define layer blocks for a simple sequential model
input_block <- function(model, input_shape) {
  keras_model_sequential(input_shape = input_shape)
}
hidden_block <- function(model, units = 32) {
  model |> layer_dense(units = units, activation = "relu")
}
output_block <- function(model, num_classes) {
  model |> layer_dense(units = num_classes, activation = "softmax")
}
```

Next, the **parsnip** model specification is created using `create_keras_sequential_spec()`:

```
# Create the model specification
create_keras_sequential_spec(
  model_name = "my_iris_mlp",
  layer_blocks = list(
    input = input_block,
    hidden = hidden_block,
    output = output_block
  ),
  mode = "classification"
)
```

The `my_iris_mlp()` function can now be used as a **parsnip** model.

```
# Define the model and fit it
iris_fit <- my_iris_mlp(
  num_hidden = 2,
  hidden_units = 16,
  fit_epochs = 10
) |>
  set_engine("keras") |>
  fit(Species ~ ., data = iris)
```

This simple example demonstrates how **kerasnip** can be used to quickly create and train a **keras3** model within the **tidymodels** framework.

5 Case study: Predicting house prices with a multi-input functional model

To demonstrate the capabilities of **kerasnip**, the Ames Housing dataset will be used to predict house prices. This case study highlights a complete **tidymodels** workflow for a regression task using a **keras3** functional model with multiple inputs, where numerical and categorical features are processed through separate input branches.

5.1 Data Preparation

First, the necessary packages are loaded and the data prepared. The Ames Housing dataset from the **modeldata** package will be used, relevant columns selected, and the data split into training and testing sets.

```
library(kerasnip)
library(tidymodels)
library(keras3)
library(dplyr) # For data manipulation
library(ggplot2) # For plotting
library(finetune) # For racing
library(workflowsets) # For comparing models

# Select relevant columns and remove rows with missing values
ames_df <- ames |>
  select(
    Sale_Price,
    Gr_Liv_Area,
    Year_Built,
    Neighborhood,
    Bldg_Type,
    Overall_Cond,
    Total_Bsmt_SF,
    contains("SF")
  ) |>
  na.omit()

# Split data into training and testing sets
set.seed(123)
ames_split <- initial_split(ames_df, prop = 0.8, strata = Sale_Price)
ames_train <- training(ames_split)
ames_test <- testing(ames_split)

# Create cross-validation folds for tuning
ames_folds <- vfold_cv(ames_train, v = 5, strata = Sale_Price)
```

To gain initial insights into the dataset, Figure **??** scatters `Sale_Price` against `Gr_Liv_Area`, illustrating the relationship between these key variables.

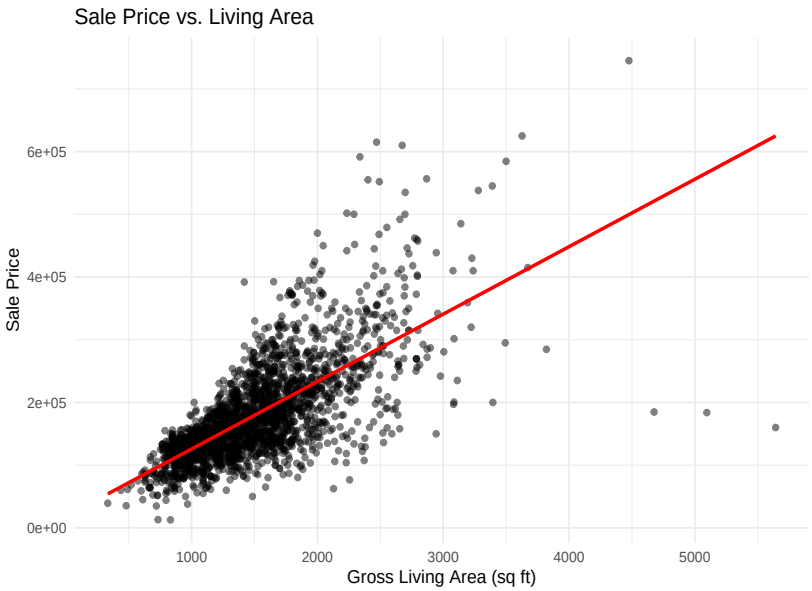

Figure 2: Data Exploration Plot for Ames Housing Dataset

5.2 Recipe for Preprocessing

A **recipes** object is created to normalize numerical predictors, one-hot encode categorical predictors, and then collapse each group of predictors (e.g., all dummy variables associated with a factor) into a single matrix column via `step_collapse()`. This preparation is crucial for multi-input **keras3** models because every branch in the functional API expects a single tensor as input.

```
ames_recipe <- recipe(Sale_Price ~ ., data = ames_train) |>
  step_normalize(all_numeric_predictors()) |>
  step_collapse(
    all_numeric_predictors(),
    new_col = "numerical_input"
  ) |>
  step_dummy(Neighborhood) |>
  step_collapse(
    starts_with("Neighborhood"),
    new_col = "neighborhood_input"
  ) |>
  step_dummy(Bldg_Type) |>
  step_collapse(
    starts_with("Bldg_Type"),
    new_col = "bldg_input"
  ) |>
  step_dummy(Overall_Cond) |>
  step_collapse(
    starts_with("Overall_Cond"),
    new_col = "condition_input"
  )
```

5.3 Define Keras Functional Model with kerasnip

Next, the Keras functional model is defined using **kerasnip**'s layer blocks. This model will have four distinct input layers: one for numerical features and three for categorical features. These branches will be processed separately and then concatenated before the final output layer.

Define input layer blocks:

```
# Define input layer blocks for multi-input functional model.
# Each function represents an input block in the Keras model graph.
input_numerical <- function(input_shape) {
  layer_input(shape = input_shape, name = "numerical_input")
```

```
}
input_neighborhood <- function(input_shape) {
  layer_input(shape = input_shape, name = "neighborhood_input")
}
input_bldg <- function(input_shape) {
  layer_input(shape = input_shape, name = "bldg_input")
}
input_condition <- function(input_shape) {
  layer_input(shape = input_shape, name = "condition_input")
}
```

Then, the dense layer blocks that will process the numerical and categorical inputs are defined:

```
# Define dense layer blocks for processing numerical and categorical inputs.
dense_numerical <- function(tensor, units = 32, activation = "relu") {
  tensor |>
    layer_dense(units = units, activation = activation)
}
dense_categorical <- function(tensor, units = 16, activation = "relu") {
  tensor |>
    layer_dense(units = units, activation = activation)
}
```

Finally, the concatenation layer block to combine the processed features and the output layer block for regression are defined:

```
# Define concatenation and output layer blocks.
concatenate_features <- function(numeric, neighborhood, bldg, condition) {
  layer_concatenate(list(numeric, neighborhood, bldg, condition))
}
dense_combined <- function(
    tensor,
    units = 64,
    activation = "relu",
    dropout = 0.2) {
  tensor |>
    layer_dense(units = units, activation = activation) |>
    layer_dropout(rate = dropout)
}
output_regression <- function(tensor) {
  layer_dense(tensor, units = 1, name = "output")
}
```

Note that `concatenate_features()` is merely a thin wrapper around `layer_concatenate()`. The helper collects the four upstream tensors into a list because `layer_concatenate()` expects a single `inputs` argument; wrapping it keeps the `layer_blocks` definition explicit while still allowing any built-in Keras layer to be used directly when its signature already matches the desired inputs.

With all the individual layer blocks defined, the **kerasnip** functional model specification is created. This specification ties together the input, processing, and output blocks, defining the flow of data through the model:

```
# Create the kerasnip model specification function.
# This function will generate a new parsnip model called
# `ames_functional_mlp`.
create_keras_functional_spec(
  model_name = "ames_functional_mlp",
  layer_blocks = list(
    # Define the four input layers of the model.
    numerical_input = input_numerical,
    neighborhood_input = input_neighborhood,
    bldg_input = input_bldg,
    condition_input = input_condition,
    # Process each input branch with a dense layer.
```

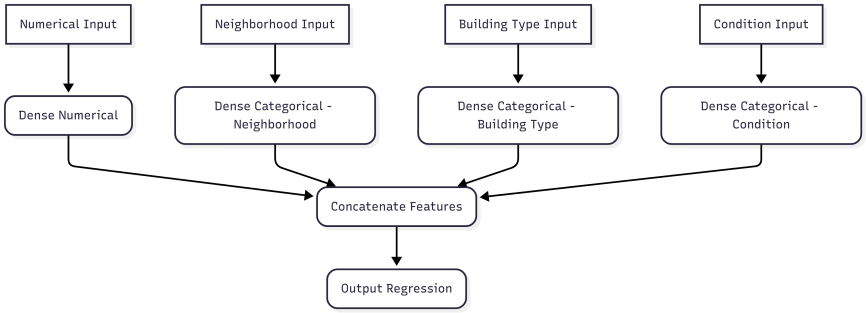

Figure 3: Functional Model Architecture for Ames Housing Dataset

```
    # `inp_spec()` defines the input tensor for each block.
    processed_numerical = inp_spec(dense_numerical, "numerical_input"),
    processed_neighborhood = inp_spec(
      dense_categorical,
      "neighborhood_input"
    ),
    processed_bldg = inp_spec(dense_categorical, "bldg_input"),
    processed_condition = inp_spec(dense_categorical, "condition_input"),
    # Concatenate the processed branches.
    # The named character vector in `inp_spec()` lists the argument names of
    # `concatenate_features` and assigns each to the upstream block that
    # supplies its tensor.
    combined_features = inp_spec(
      concatenate_features,
      c(
        numeric = "processed_numerical",
        neighborhood = "processed_neighborhood",
        bldg = "processed_bldg",
        condition = "processed_condition"
      )
    ),
    # Add a dense layer after concatenation
    processed_combined = inp_spec(dense_combined, "combined_features"),
    # Define the final output layer.
    output = inp_spec(output_regression, "processed_combined")
  ),
  mode = "regression"
)
```

Figure **??** visualizes the resulting graph:

5.4 Model Specification and Workflow

The `ames_functional_mlp` model specification is defined, setting some hyperparameters to `tune()`. Then a `workflow` that combines the recipe and the model specification is created.

```
# Define the tunable model specification.
# The `ames_functional_mlp()` function that was created is used.
# The arguments of the layer blocks can be tuned by prefixing them with the
# block name (e.g., `processed_numerical_units`).
functional_mlp_spec <- ames_functional_mlp(
  processed_numerical_units = tune(id = "numerical_units"),
  processed_neighborhood_units = 256,
  processed_bldg_units = 256,
  processed_condition_units = 256,
  num_processed_combined = tune(id = "num_combined_layers"),
  processed_combined_units = tune(id = "combined_units"),
  processed_combined_dropout = tune(id = "combined_dropout"),
  # compile and fit arguments can also be passed for the \CRANpkg{keras3} model.
```

```
  compile_loss = "mean_squared_error",
  compile_optimizer = tune(),
  compile_metrics = c("mean_absolute_error"),
  learn_rate = tune(),
  fit_epochs = 100,
  fit_batch_size = 32,
  fit_validation_split = 0.2,
  fit_callbacks = list(
    callback_early_stopping(monitor = "val_loss", patience = 10)
  )
) |>
  set_engine("keras")

# Create a workflow with the recipe and the model specification.
ames_wf <- workflow() |>
  add_recipe(ames_recipe) |>
  add_model(functional_mlp_spec)
```

5.5 Tune Model

A tuning grid is defined and `tune_race_anova()` is used to perform cross-validation and find the best hyperparameters.

```
# Define the tuning grid
params <- extract_parameter_set_dials(ames_wf) |>
  update(
    numerical_units = hidden_units(range = c(64, 128)),
    num_combined_layers = num_terms(range = c(3, 5)),
    combined_units = hidden_units(range = c(64, 256)),
    combined_dropout = dropout(range = c(0.1, 0.5)),
    learn_rate = learn_rate(range = c(0.01, 0.05), trans = NULL),
    compile_optimizer = optimizer_function(
      values = c("adam", "rmsprop")
    )
  )

functional_mlp_grid <- grid_regular(params, levels = 2)

ames_tune_results <- tune_race_anova(
  ames_wf,
  resamples = ames_folds,
  grid = functional_mlp_grid,
  metrics = metric_set(rmse, mae, rsq),
  control = control_race(save_pred = TRUE, save_workflow = TRUE)
)
```

5.6 Visualizing Tuning Results

Table **??** reports the learning rate, optimizer, and number of combined layers for the best-performing configurations.

5.7 Comparing Models with workflowsets

To demonstrate the flexibility of **kerasnip** within the broader **tidymodels** ecosystem, `ames_functional_mlp` can be compared with a traditional statistical model using **workflowsets**. This allows for a systematic comparison of different modeling approaches on the same dataset.

First, a simple linear regression model and a corresponding recipe are defined.

```
# Define a linear regression model for comparison
linear_reg_spec <- linear_reg() |>
  set_engine("lm")
```

Table 1: Tuning Results Summary (Top RMSE Candidates)

learning_rate	optimizer	num_units	combined_layers	comb_units	comb_dropout	rmse
0.01	adam	64	3	64	0.1	37595.64
		64	3	64	0.5	39586.93
		64	3	256	0.1	35693.79
		64	3	256	0.5	43446.65
		64	5	64	0.1	41047.63
		64	5	256	0.1	36576.71
		64	5	256	0.5	42987.74
		128	3	64	0.1	37038.40
		128	3	64	0.5	42904.82
		128	3	256	0.1	37546.45
		128	3	256	0.5	36806.73
		128	5	64	0.1	39004.30
		128	5	256	0.1	36746.76
	rmsprop	64	3	64	0.1	43236.38
		64	3	64	0.5	46536.41
		64	3	256	0.5	42205.63
		64	5	256	0.1	45557.59
		128	3	64	0.1	38122.71
		128	3	64	0.5	40911.50
		128	5	64	0.1	45079.44
		128	5	256	0.1	35540.96
0.05	adam	64	3	64	0.1	39109.81
		64	3	64	0.5	46394.76
		64	3	256	0.1	36851.49
		64	3	256	0.5	41796.49
		128	3	64	0.1	39974.52
		128	3	64	0.5	47511.90
		128	3	256	0.1	32858.04
	rmsprop	128	3	64	0.1	47010.48

```
# Create a simpler recipe for the linear regression model
# (without step_collapse, as it's not needed for single-input models)
linear_reg_recipe <- recipe(Sale_Price ~ ., data = ames_train) |>
  step_normalize(all_numeric_predictors()) |>
  step_dummy(all_nominal_predictors())
```

Next, a `workflowset` containing both the **kerasnip** workflow and the linear regression workflow is created.

```
ames_workflowset <- workflow_set(
  preproc = list(
    kerasnip = ames_recipe,
    linear_reg = linear_reg_recipe
  ),
  models = list(
    kerasnip_mlp = functional_mlp_spec,
    linear_model = linear_reg_spec
  )
)

# Combine the workflows into a workflowset
ames_workflowset <- ames_workflowset |>
  filter(
    wflow_id %in% c("kerasnip_kerasnip_mlp", "linear_reg_linear_model")
  ) |>
  option_add(grid = functional_mlp_grid, id = "kerasnip_kerasnip_mlp") |>
  option_add(grid = 10, id = "linear_reg_linear_model")
```

Now, the `workflowset` can be tuned using `workflow_map()`.

```
ames_workflowset_results <- ames_workflowset |>
  workflow_map(
    "tune_race_anova",
    resamples = ames_folds,
```

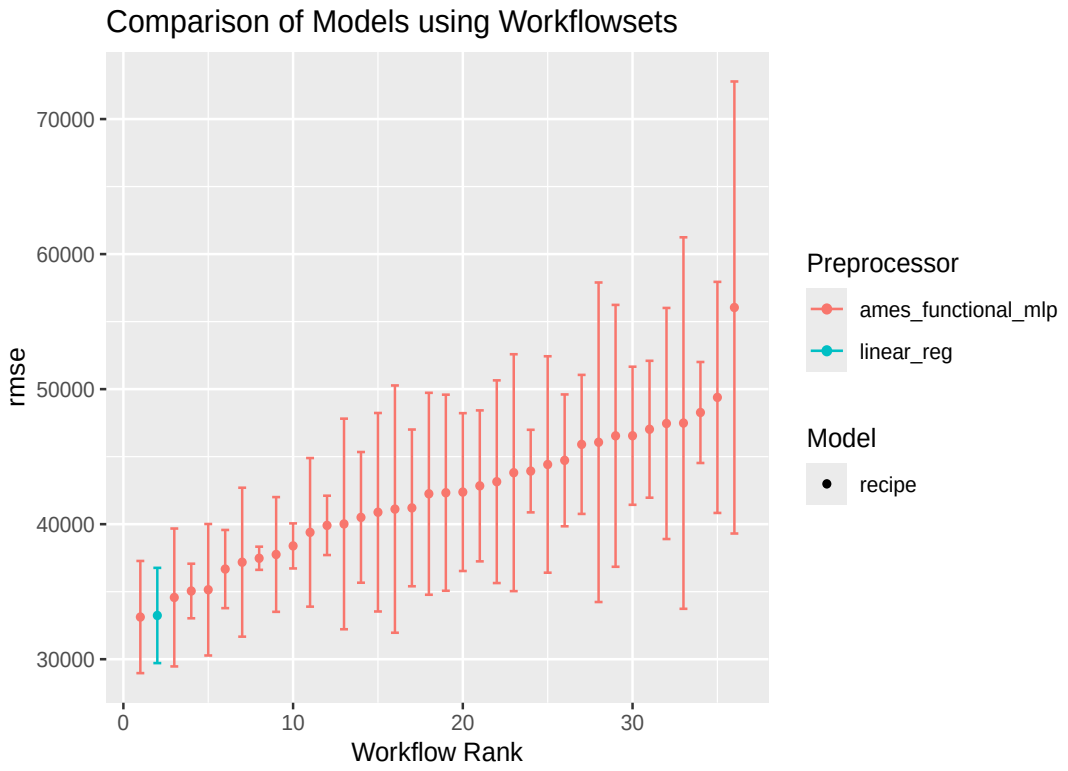

Figure 4: Workflowset Results - RMSE across Different Models

```
    metrics = metric_set(rmse, mae, rsq),
    control = control_race(save_pred = TRUE, save_workflow = TRUE)
  )
```

Figure **??** visualizes the `workflowset` comparison so the relative performance of the baseline linear model and the functional MLP can be assessed side by side.

5.8 Finalize Workflow and Evaluate

```
best_functional_mlp_params <- select_best(ames_tune_results, metric = "rmse")

final_ames_wf <- finalize_workflow(ames_wf, best_functional_mlp_params)

final_ames_fit <- fit(final_ames_wf, data = ames_train)

ames_test_pred <- predict(
  final_ames_fit,
  new_data = ames_test
)
```

The performance of the final model on the test set is shown in the following table.

5.9 Visualizing Model Performance

Figure **??** contrasts predictions with observed prices to illustrate how well the tuned workflow generalizes to unseen data.

This case study demonstrates how **kerasnip** can be used to define, tune, and evaluate a complex, multi-input **keras3** model within a standard **tidymodels** workflow. The ability to seamlessly integrate such models opens up new possibilities for tackling complex machine learning problems in R.

wflow_id	.metric	mean	std_err	.config
kerasnip_kerasnip_mlp	rmse	33122.90	2525.470	pre0_mod11_post0
linear_reg_linear_model	rmse	33238.93	2144.440	pre0_mod0_post0
kerasnip_kerasnip_mlp	rmse	34572.16	3104.013	pre0_mod41_post0
kerasnip_kerasnip_mlp	rmse	35051.24	1228.393	pre0_mod57_post0
kerasnip_kerasnip_mlp	rmse	35144.00	2959.411	pre0_mod25_post0
kerasnip_kerasnip_mlp	rmse	36676.42	1759.823	pre0_mod01_post0
kerasnip_kerasnip_mlp	rmse	37184.60	3352.894	pre0_mod27_post0
kerasnip_kerasnip_mlp	rmse	37476.51	521.139	pre0_mod19_post0
kerasnip_kerasnip_mlp	rmse	37757.21	2583.135	pre0_mod09_post0
kerasnip_kerasnip_mlp	rmse	38390.95	1014.105	pre0_mod17_post0
kerasnip_kerasnip_mlp	rmse	39395.43	3345.039	pre0_mod03_post0
kerasnip_kerasnip_mlp	rmse	39909.96	1335.792	pre0_mod12_post0
kerasnip_kerasnip_mlp	rmse	40016.36	4740.274	pre0_mod26_post0
kerasnip_kerasnip_mlp	rmse	40504.98	2942.000	pre0_mod05_post0
kerasnip_kerasnip_mlp	rmse	40883.88	4468.054	pre0_mod49_post0
kerasnip_kerasnip_mlp	rmse	41116.34	5566.620	pre0_mod02_post0
kerasnip_kerasnip_mlp	rmse	41205.46	3528.358	pre0_mod29_post0
kerasnip_kerasnip_mlp	rmse	42250.80	4545.248	pre0_mod15_post0
kerasnip_kerasnip_mlp	rmse	42328.54	4414.458	pre0_mod50_post0
kerasnip_kerasnip_mlp	rmse	42376.67	3553.134	pre0_mod18_post0
kerasnip_kerasnip_mlp	rmse	42837.85	3398.218	pre0_mod14_post0
kerasnip_kerasnip_mlp	rmse	43145.28	4562.742	pre0_mod33_post0
kerasnip_kerasnip_mlp	rmse	43812.70	5332.857	pre0_mod10_post0
kerasnip_kerasnip_mlp	rmse	43932.14	1858.352	pre0_mod13_post0
kerasnip_kerasnip_mlp	rmse	44420.38	4872.205	pre0_mod45_post0
kerasnip_kerasnip_mlp	rmse	44726.03	2968.202	pre0_mod22_post0
kerasnip_kerasnip_mlp	rmse	45907.85	3129.222	pre0_mod21_post0
kerasnip_kerasnip_mlp	rmse	46066.50	7192.835	pre0_mod04_post0
kerasnip_kerasnip_mlp	rmse	46541.57	5894.981	pre0_mod08_post0
kerasnip_kerasnip_mlp	rmse	46548.22	3108.154	pre0_mod51_post0
kerasnip_kerasnip_mlp	rmse	47028.88	3081.750	pre0_mod30_post0
kerasnip_kerasnip_mlp	rmse	47456.52	5201.905	pre0_mod31_post0
kerasnip_kerasnip_mlp	rmse	47488.55	8362.792	pre0_mod34_post0
kerasnip_kerasnip_mlp	rmse	48270.46	2273.806	pre0_mod06_post0
kerasnip_kerasnip_mlp	rmse	49387.63	5202.067	pre0_mod46_post0
kerasnip_kerasnip_mlp	rmse	56045.59	10175.920	pre0_mod16_post0

.metric	.estimator	.estimate
rmse	standard	36820.125073
mae	standard	21806.668965
rsq	standard	0.841205

Figure 5: Predicted vs. Actual Sale Prices on the Test Set

compile_optimizer	learn_rate	units nu | com	combined_dropout	num_combined_layers	mean	std_err
adam	0.05	128 | 256	0.1	3	32858.04	2087.124
rmsprop	0.01	128 | 256	0.1	5	35540.96	3230.442
adam	0.01	64 | 256	0.1	3	35693.79	1521.243
adam	0.01	64 | 256	0.1	5	36576.71	3205.070
adam	0.01	128 | 256	0.1	5	36746.76	5275.331

6 Conclusion

In this paper, **kerasnip** was introduced as an R package that bridges **keras3** and **tidymodels**. The examples and the Ames Housing case study demonstrated how practitioners can define, preprocess, tune, and evaluate complex **keras3** models inside reproducible **tidymodels** workflows. **kerasnip**'s core innovation lies in dynamic code generation, enabling architectural tuning and democratizing deep learning access in R while remaining faithful to **tidyverse** design principles. Future work will focus on shipping higher-level helpers for popular architectures (e.g., CNN templates, Transformer scaffolds), enhancing **tfdatasets** integration, and smoothing workflows for fine-tuning pre-trained **keras3** applications—capabilities that build on top of, rather than unlock, the flexible building blocks already present. **kerasnip** is envisioned as a cornerstone for the R deep learning community, fostering innovation.

References

M. Abadi, A. Agarwal, P. Barham, E. Brevdo, Z. Chen, C. Citro, G. S. Corrado, A. Davis, J. Dean, M. Devin, S. Ghemawat, I. Goodfellow, A. Harp, G. Irving, M. Isard, Y. Jia, R. Jozefowicz, L. Kaiser, M. Kudlur, J. Levenberg, D. Mané, R. Monga, S. Moore, D. Murray, C. Olah, M. Schuster, J. Shlens, B. Steiner, I. Sutskever, K. Talwar, P. Tucker, V. Vanhoucke, V. Vasudevan, F. Viégas, O. Vinyals, P. Warden, M. Wattenberg, M. Wicke, Y. Yu, and X. Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. URL http://tensorflow.org/. Software available from tensorflow.org. [p]

J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang. JAX: composable transformations of Python+NumPy programs, 2018. URL http://github.com/google/jax. [p]

F. Chollet. Keras. https://github.com/fchollet/keras, 2015. [p]

D. Falbel. *luz: Higher-Level Interface for torch*, 2023. URL https://github.com/mlverse/luz. R package version 0.3.0. [p]

I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016. http://www.deeplearningbook.org. [p]

M. Kuhn and H. Wickham. Tidymodels: a framework for modeling and machine learning in R. *The R Journal*, 2020. URL https://tidymodels.org. R package version 0.1.0. [p]

A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems 32*, pages 8024–8035. Curran Associates, Inc., 2019. URL http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf. [p]

K. Ushey, J. J. Allaire, and Y. Tang. *reticulate: R Interface to Python*, 2023. URL https://CRAN.R-project.org/package=reticulate. R package version 1.30. [p]

David Díaz
Independent Researcher

ORCiD: 0000-0002-0927-9795
daviddrsch@gmail.com